

Explicación Práctica de Monitores

Ejercicios

EJERCICIO 4

En un banco hay 3 empleados. Y hay N clientes que deben ser atendidos por uno de ellos (cualquiera) de acuerdo al orden de llegada. Cuando uno de los empleados lo atiende el cliente le entrega los papeles y espera el resultado.

En este caso el cliente debe entrar al banco y esperar a que le llegue su turno, en ese momento se le indica a que empleado debe ir y debe esperar a que este lo atienda.

¿Que monitores y procesos se usaran?

Posible solución:

- **Debemos tener los procesos clientes y los empleados.**
- **Por otro lado debemos tener un monitor para administrar la entrada (ordenar a los clientes y asignarle un empleado).**
- **Para realizar la interacción entre un empleado y un cliente podemos tener un monitor para cada empleado.**



EJERCICIO 4

Primero vamos a ver la parte del problema donde los clientes llegan al banco y esperan su turno de atención. Luego vemos la interacción entre el cliente y el empleado.

El cliente llega al banco y si hay algún empleado libre **NO** debe esperar y directamente elige a uno de ellos para que lo atienda; sino se duerme y espera a que algún empleado que quede libre lo despierte. Para esto el cliente llama al procedimiento *Llegada* que tiene como parámetro de salida el empleado que va a atender al cliente.

```
Process Cliente[id: 0..N-1]
{ int idE;
  Banco.llegada(idE);
  ....
}
```

El empleado continuamente está atendiendo a los clientes. Cuando está libre avisa al banco que espera un próximo cliente llamando al procedimiento *Próximo* enviando como parámetro su identificador. Si hay algún cliente esperando lo despierta al primero que está en la cola.

```
Process Empleado[id: 0..2]
{ while (true)
  { Banco.próximo(id);
    ....
  }
}
```



EJERCICIO 4

Vamos a usar una cola donde guardaremos los identificadores de los empleados que están libres llamada *elibres*. También usaremos una variable *condition* llamada *esperaC* donde se demoran los clientes cuando no hay empleados libres, y un entero *esperando* para contar la cantidad de clientes que están dormidos (como se vio en el ejercicio 2).

Al modificar la cola *elibres* agregando al empleado libre, cualquier cliente que no estaba dormido puede verlo, pasar y sacar a ese empleado de la cola (con la posibilidad de dejarla vacía). Cuando el que fue despertado accede al monitor hace el pop de una cola vacía → NO SE RESPETA EL ORDEN Y DA ERROR

Monitor Banco {

```
cola elibres;  
cond esperaC;  
int esperando = 0;
```

Procedure Llegada(idE: out int)

```
{ if (empty(elibres)) { esperando ++;  
                        wait (esperaC);  
                        }  
  pop(elibres, idE);  
}
```

Procedure Próximo(idE: in int)

```
{ push(elibres, idE);  
  if (esperando > 0 ) { esperando --;  
                      signal (esperaC);  
                      }  
}
```



EJERCICIO 4

Para evitar el problema anterior se debe usar *passing the condition*, pero el empleado debe insertar su ID en la cola haya o no clientes dormidos → la condición a chequear por el cliente no debe ser la cola *elibres*.

Agregamos un entero *cantLibres* que llevará la cuenta de los empleados que están “realmente” libres (llamaron al procedimiento *próximo* cuando no había nadie esperando). No siempre su valor será equivalente a la cantidad de elementos de *elibres*.

Esta nueva variable será sobre la que se hace el *passing the condition*.

Monitor Banco {

```
cola elibres;  
cond esperaC;  
int esperando = 0, cantLibres = 0;
```

Procedure Llegada(idE: out int)

```
{ if (cantLibres == 0) { esperando ++;  
                           wait (esperaC);  
                           }
```

```
    else cantLibres--;
```

```
    pop(elibres, idE);
```

```
}
```

Procedure Próximo(idE: in int)

```
{ push(elibres, idE);  
  if (esperando > 0) { esperando --;  
                      signal (esperaC);  
                      }
```

```
    else cantLibres++;
```

```
}
```

```
}
```

EJERCICIO 4

Ahora vemos la interacción entre el cliente y el empleado que lo debe atender. Para esto tendremos un monitor *escritorio* para cada empleado que es donde interactúan

El cliente, cuando ya sabe a que empleado ir se dirige a su escritorio, le entrega los papeles y espera a que el empleado le retorne los resultados por medio del procedimiento *atención*.

Process Cliente[id: 0..N-1]

```
{ int idE;  
  text papel, res;  
  
  Banco.llegada(idE);  
  Escritorio[idE].atención(papel, res);  
}
```

El empleado, después de avisar que está libre, se dirige a su escritorio a esperar a un cliente llamando al procedimiento *esperarDatos* con un parámetro de salida donde recibe los papeles del cliente.

Resuelve la solicitud y le envía los resultados al cliente por medio del procedimiento *enviarResultado*, y espera a que el cliente tome los resultados y se vaya para poder atender a otro.

Process Empleado[id: 0..2]

```
{ text datos;  
  
  while (true)  
  { Banco.próximo(id);  
    Escritorio[id].esperarDatos(datos);  
    res = resolver solicitud en base a datos  
    Escritorio[id].enviarResultado(res);  
  }  
}
```



EJERCICIO 4

En cada escritorio debemos tener una variable para que el cliente le comunique al empleado los papeles o datos y otra para comunicarse los resultados en el sentido contrario.

Además el cliente debe esperar hasta obtener los resultados en una variable condición *vcCliente*. El empleado debe esperar los datos en otra variable condición *vcEmpleado*, y luego debe esperar a que el cliente haya tomado los resultados, para lo cual podemos usar esa misma variable condición.

```
Monitor Escritorio[id: 0..2] {  
  cond vcCliente, vcEmpleado;  
  text datos, resultados;
```

```
  Procedure Atención(D: in text; R: out text)
```

```
  { datos = D;  
    signal (vcEmpleado);  
    wait (vcCliente);  
    R = resultados;  
    signal (vcEmpleado);  
  }
```

Avisa que ya dejó los datos

Debe esperar los resultados

Avisa que ya tomó los datos

```
  Procedure Esperardatos(D: out text)
```

```
  { wait (vcEmpleado);  
    D = datos;  
  }
```

Espera que llegue el cliente

Y si el cliente ya había llegado

```
  Procedure EnviarResultados(R: in text)
```

```
  { resultados = R;  
    signal (vcCliente);  
    wait (vcEmpleado);  
  }
```

Avisa que están los resultados

Debe esperar que el cliente tome los resultados

```
}
```

EJERCICIO 4

Si el cliente llega al escritorio antes que el empleado entonces el empleado se duerme y nadie lo despierta (el cliente ya hizo el *signal* y no tiene efecto posterior) → DEADLOCK.

Debo usar una variable booleana *listo* que me indique si el cliente ya llegó, y sólo dormirse si *listo* es falso.

```
Monitor Escritorio[id: 0..2] {  
  cond vcCliente, vcEmpleado;
```

```
  text datos, resultados;
```

```
  boolean listo = false;
```

```
  Procedure Atención(D: in text; R: out text)
```

```
  { datos = D;
```

```
    listo = true;
```

Marca que llegó el cliente

```
    signal (vcEmpleado);
```

```
    wait (vcCliente);
```

```
    R = resultados;
```

```
    signal (vcEmpleado);
```

```
  }
```

```
  Procedure Esperardatos(D: out text)
```

```
  { if (not listo) wait (vcEmpleado);
```

```
    D = datos;
```

Sólo se duerme si el cliente no llegó.

```
  }
```

```
  Procedure EnviarResultados(R: in text)
```

```
  { resultados = R;
```

```
    signal (vcCliente);
```

```
    wait (vcEmpleado);
```

```
    listo = false;
```

Resetea para atender al próximo cliente

```
  }
```

```
}
```


EJERCICIO 4

Process Cliente[id: 0..N-1]

```
{ int idE;
  text papel, res;
  Banco.llegada(idE);
  Escritorio[idE].atención(papel, res);
}
```

Process Empleado[id: 0..2]

```
{ text datos;
  while (true)
  { Banco.próximo(id);
    Escritorio[id].esperarDatos(datos);
    res = resolver solicitud en base a datos;
    Escritorio[id].enviarResultado(res);
  }
}
```

Monitor Banco {

```
cola elibres;
cond esperaC;
int esperando = 0, cantLibres = 0;
```

Procedure Llegada(idE: out int)

```
{ if (cantLibres == 0)
  { esperando ++;
    wait (esperaC);
  }
  else cantLibres--;
  pop(elibres, idE);
}
```

Procedure Próximo(idE: in int)

```
{ push(elibres, idE);
  if (esperando > 0 )
  { esperando --;
    signal (esperaC);
  }
  else cantLibres++;
}
```

Monitor Escritorio[id: 0..2] {

```
cond vcCliente, vcEmpleado;
text datos, resultados;
boolean listo = false;
```

Procedure Atención(D: in text; R: out text)

```
{ datos = D;
  listo = true;
  signal (vcEmpleado);
  wait (vcCliente);
  R = resultados;
  signal (vcEmpleado);
}
```

Procedure Esperardatos(D: out text)

```
{ if (not listo) wait (vcEmpleado);
  D = datos;
}
```

Procedure EnviarResultados(R: in text)

```
{ resultados = R;
  signal (vcCliente);
  wait (vcEmpleado);
  listo = false;
}
```

```
}
```

EJERCICIO 5

Se debe simular un partido de fútbol 11. Cuando los 22 jugadores llegaron a la cancha juegan durante 90 minutos y luego todos se retiran.

En este caso se debe hacer una barrera hasta que llegan los 22 jugadores, y luego **TODOS JUNTOS AL MISMO TIEMPO** deben jugar durante 90 minutos.

Tenemos un contador que se incrementa al llegar cada jugador y se duermen en una variable condición hasta que llega el último y los despierta para jugar.

Process Jugador[id: 0..21]

```
{ Cancha.llegada();  
  delay (90minutos); //juega el partido  
}
```

Cada jugador juega su propio partido posiblemente en diferentes momentos

Monitor Cancha

```
{ int cant = 0;  
  cond espera;
```

Procedure llegada ()

```
{ cant ++;  
  if (cant < 22) wait (espera)  
  else signal_all (espera);  
}
```

EJERCICIO 5

El *delay* que representa que todos están jugando al mismo tiempo el partido se debe hacer en un único lugar. Podría ser en el monitor, cuando llega el último jugador (antes del `signal_all`). O bien usar otro proceso que simula el partido y se duerme hasta que todos llegan; luego hace el *delay* y despierta a todos para que se vayan.

```
Process Jugador[id: 0..21]  
{ Cancha.llegada();  
}
```

```
Process Partido  
{ Cancha.Iniciar();  
  delay (90minutos); // se juega el partido  
  Cancha.Terminar();  
}
```

Monitor Cancha

```
{ int cant = 0;  
  cond espera, inicio;
```

Procedure llegada ()

```
{ cant ++;  
  if (cant < 22) wait (espera)  
  else signal (inicio);  
}
```

Procedure Iniciar ()

```
{ if (cant < 22) wait (inicio);  
}
```

Procedure Terminar ()

```
{ signal_all(espera);  
}  
}
```

El último en llegar despierta al *partido*.

Sólo se duerme si no han llegado todos.

Despierta a todos para que se vayan.

EJERCICIO 5

```
Process Jugador[id: 0..21]  
{ Cancha.llegada();  
}
```

```
Process Partido  
{ Cancha.Iniciar();  
  delay (90minutos); // se juega el partido  
  Cancha.Terminar();  
}
```

Monitor Cancha

```
{ int cant = 0;  
  cond espera, inicio;
```

Procedure llegada ()

```
{ cant ++;  
  if (cant < 22) wait (espera)  
  else signal (inicio);  
}
```

Ese Jugador se ira sin Jugar

Procedure Iniciar ()

```
{ if (cant < 22) wait (inicio);  
}
```

Procedure Terminar ()

```
{ signal_all(espera);  
}  
}
```

El último jugador en llegar “Inicia” el partido, pero no se duerme (se va sin jugar). Debemos hacer que él también se duerma en *espera*, al igual que el resto de los jugadores.



EJERCICIO 5

```
Process Jugador[id: 0..21]  
{ Cancha.llegada();  
}
```

```
Process Partido  
{ Cancha.Iniciar();  
  delay (90minutos); // se juega el partido  
  Cancha.Terminar();  
}
```

Monitor Cancha

```
{ int cant = 0;  
  cond espera, inicio;  
  
  Procedure llegada ()  
  { cant ++;  
    if (cant == 22) signal (inicio);  
    wait (espera);  
  }  
  
  Procedure Iniciar ()  
  { if (cant < 22) wait (inicio);  
  }  
  
  Procedure Terminar ()  
  { signal_all(espera);  
  }  
}
```



EJERCICIO de la práctica

En un entrenamiento de futbol hay 20 jugadores que forman 4 equipos (cada jugador conoce el equipo al cual pertenece llamando a la función `DarEquipo()`). Cuando un equipo está listo (han llegado los 5 jugadores que lo componen), debe enfrentarse a otro equipo que también esté listo (los dos primeros equipos en juntarse juegan en la cancha 1, y los otros dos equipos juegan en la cancha 2). Una vez que el equipo conoce la cancha en la que juega, sus jugadores se dirigen a ella. Cuando los 10 jugadores del partido llegaron a la cancha comienza el partido, juegan durante 50 minutos, y al terminar todos los jugadores del partido se retiran (no es necesario que se esperen para salir).



EJERCICIO de la práctica

Process Jugador[id: 0..19]

```
{ Cancha.llegada();  
}
```

¿Como sabe a que cancha ir?

Process Partido [id: 0..1]

```
{ Cancha[id].Iniciar();  
  delay (90minutos); // se juega el partido  
  Cancha[id].Terminar();  
}
```

Monitor Cancha [id: 0..1]

```
{ int cant = 0;  
  cond espera, inicio;
```

Procedure llegada ()

```
{ cant ++;  
  if (cant == 10) signal (inicio);  
  wait (espera);  
}
```

Procedure Iniciar ()

```
{ if (cant < 10) wait (inicio);  
}
```

Procedure Terminar ()

```
{ signal_all(espera);  
}  
}
```



EJERCICIO de la práctica

Primero cada equipo se debe juntar (independientemente del resto), y luego debe ver a que cancha ir. Se necesita un monitor para cada equipo.

Process Jugador[id: 0..19]

```
{ int miEquipo = ...;
  int numeroC;

  Equipo[miEquipo].llegada(numeroC);
}
```

Monitor Equipo [id: 0..3]

```
{ int cant = 0;
  cond espera;

  Procedure llegada (cancha: OUT int)
  { cant ++;
    if (cant < 4) wait (espera)
    else { cancha = Determinar la cancha
          signal_all (espera);
        }
  }
}
```

¿Cómo se hace, porque no depende sólo de este grupo?



EJERCICIO de la práctica

Se requiere un monitor para administrar las canchas

Process Jugador[id: 0..19]

```
{ int miEquipo = ...;
  int numeroC;

  Equipo[miEquipo].llegada(numeroC);
}
```

Monitor Admin

```
{ int cant = 0;

  Procedure DarCancha (cancha: OUT int)
  { cant ++;
    if (cant <= 2) cancha = 0
    else cancha = 1;
  }
}
```

Monitor Equipo [id: 0..3]

```
{ int cant = 0;
  cond espera;

  Procedure llegada (cancha: OUT int)
  { cant ++;
    if (cant < 4) wait (espera)
    else { Admin.DarCancha(cancha);
          signal_all (espera);
        }
  }
}
```

Sólo lo conocerá uno de los integrantes del grupo (el último en llegar)



EJERCICIO de la práctica

Process Jugador[id: 0..19]

```
{ int miEquipo = ...;
  int numeroC;

  Equipo[miEquipo].llegada(numeroC);
}
```

Monitor Admin

```
{ int cant = 0;

  Procedure DarCancha (cancha: OUT int)
  { cant ++;
    if (cant <= 2) cancha = 0
    else cancha = 1;
  }
}
```

Monitor Equipo [id: 0..3]

```
{ int cant = 0, numCancha;
  cond espera;

  Procedure llegada (cancha: OUT int)
  { cant ++;
    if (cant < 4) wait (espera)
    else { Admin.DarCancha(numCancha);
          signal_all (espera);
        };
    cancha = numCancha;
  }
}
```

Ahora si cada persona puede ir a la cancha que le corresponde y jugar el partido.



EJERCICIO de la práctica

Process Jugador[id: 0..19]

```
{ int miEquipo = ...;
  int numeroC;

  Equipo[miEquipo].llegada(numeroC);
  Cancha[numeroC].llegada();
}
```

Process Partido [id: 0..1]

```
{ Cancha[id].Iniciar();
  delay (90minutos); // se juega el partido
  Cancha[id].Terminar();
}
```

Monitor Cancha [id: 0..1]

```
{ int cant = 0;
  cond espera, inicio;

  Procedure llegada ()
  { cant ++;
    if (cant == 10) signal (inicio);
    wait (espera);
  }
```

Procedure Iniciar ()

```
{ if (cant < 10) wait (inicio);
}
```

Procedure Terminar ()

```
{ signal_all(espera);
}
}
```



EJERCICIO de la práctica

Process Jugador[id: 0..19]

```
{ int miEquipo = ...;
  int numeroC;

  Equipo[miEquipo].llegada(numeroC);
  Cancha[numeroC].llegada();
}
```

Monitor Equipo [id: 0..3]

```
{ int cant = 0, numCancha;
  cond espera;
```

Procedure llegada (cancha: OUT int)

```
{ cant ++;
  if (cant < 4) wait (espera)
  else { Admin.DarCancha(numCancha);
        signal_all (espera);
      };
  cancha = numCancha;
}
}
```

Process Partido [id: 0..1]

```
{ Cancha[id].Iniciar();
  delay (90minutos); // se juega el partido
  Cancha[id].Terminar();
}
```

Monitor Admin

```
{ int cant = 0;
```

Procedure DarCancha (num: OUT int)

```
{ cant ++;
  if (cant <= 2) num = 0
  else num = 1;
}
}
```

Monitor Cancha [id: 0..1]

```
{ int cant = 0;
  cond espera, inicio;
```

Procedure llegada ()

```
{ cant ++;
  if (cant == 10) signal (inicio);
  wait (espera);
}
```

Procedure Iniciar ()

```
{ if (cant < 10) wait (inicio);
}
```

Procedure Terminar ()

```
{ signal_all(espera);
}
}
```